TypeScript Decorators & Angular Context

Learning Objectives

- Understand what decorators are and how they work
- Create and use class decorators
- Work with method, property, and parameter decorators
- Build decorator factories for customizable decorators
- Understand decorators in Angular context
- Learn the basics of `@Component` and `@Injectable`

> **Note:** To use decorators in TypeScript, you need to enable the `experimentalDecorators` option in your `tsconfig.json` file.

What are Decorators?

Decorators are special functions that can modify or enhance classes, methods, properties, or parameters. They are attached using the `@` symbol. Think of decorators as labels that add extra behavior to your code.

Real-world analogy: Imagine decorating a plain cake. The cake is your class, and decorators are the icing, sprinkles, or candles you add to enhance it. The cake remains a cake, but decorators make it special.

Decorators are widely used in frameworks like Angular to add metadata and functionality to classes and their members.

> **Output:**
>
> **🔧 How do they actually work?**
>
> A decorator is mostly just a function call. When TypeScript compiles your code to JavaScript:
>
> 1. It finds the class/method marked with `` `@MyDecorator` ``.
> 2. It runs your `` `MyDecorator` `` function.

3. It passes the class (or method) to your function as an argument.
4. **The Return Value Matters:**
    - If your decorator returns specific objects (like a new class or a descriptor), TypeScript **replaces** the original with what you returned.
    - If it returns nothing, the original stays as is, but your "side effects" (logging, tagging) still happened.

Output:

confusing Syntax Explained

You will see some specific terms used in decorator functions. Here is what they mean:

- **target**: The class itself (or its prototype). It's the thing you are decorating.
- **propertyKey**: The name of the method or property you are decorating (e.g., "calculate", "userName").
- **descriptor**: An object that holds the behavior of a method. `descriptor.value` is the actual function code.
- **constructor**: The function that builds an object. In TypeScript, a class is really just a constructor function.
- **...args**: "Rest parameters". It collects all arguments passed to a function into an array, so you can pass them along.

Class Decorators

A class decorator is a function that is applied to a class itself. It allows you to modify the class before any object is created.

1. Basic Class Decorator (Logging)

```
// 1. The Decorator Function
function LogClass(constructor: Function) {
  console.log(`Class defined: ${constructor.name}`);
}

// 2. Applying the Decorator
@LogClass
class User {
  constructor(public name: string) {}
}

// The log happens immediately when the class is read by JavaScript,
// even before we create a user!
const user = new User("Alice");
```

Output:

```
Class defined: User
```

Modern TC39 Syntax

In the new standard, we receive the class `value` and a `context` object.

```
function LogClass(value: Function, context: ClassDecoratorContext) {
  // 'value' is the class constructor itself
  if (context.kind === "class") {
    console.log(`Class defined: ${context.name}`);
  }
}

@LogClass
class User { ... }
```

Deep Dive: The Arguments

1. value (The Target)
This represents the element being decorated.
• For `@Class` : It is the **Constructor Class**.
• For `@Method` : It is the **Function** code.
• For `@Field` : It is `undefined` (fields don't exist until the object is created).

2. context (The Metadata)
This object (typed as `ClassDecoratorContext` here) is passed by the JavaScript engine and contains details *about* the usage:

- `kind` : The type of decoration (e.g., `"class"` , `"method"` , `"field"`).
- `name` : The name of the class or member (e.g., `"User"`).
- `addInitializer(fn)` : Allows you to run code after the class/member is physically defined.
- `metadata` : A storage object to modify/read metadata (Symbol-based API).

Mechanics: Here, `LogClass` returns `void` (nothing). TypeScript executes the function to let you log the message, but since you didn't return a new class constructor, the original `User` class remains unchanged.

2. Class Decorator that Adds Data

This looks complex, but it just creates a *new* class that extends your original one to add a property.

```
// "T extends { new(...args: any[]): {} }"
// This complex bit just means: "T must be a class that can be created with 'new'"
function AddRole<T extends { new(...args: any[]): {} }>(constructor: T) {
  // We return a new class that has everything the old one had...
  return class extends constructor {
    // ...plus a new property
    role = "admin";
  };
}

@AddRole
class AdminUser {
  constructor(public name: string) {}
```

```
}

const admin = new AdminUser("SuperUser");
```

Output:

```
User: SuperUser
Role: admin
```

Modern TC39 Syntax

You still return a new class, but the type signature is cleaner.

```
function AddRole {}>(
  value: T,
  context: ClassDecoratorContext
) {
  return class extends value {
    role = "admin";
  };
}

@AddRole
class AdminUser { ... }
```

Mechanics: Because `AddRole` returns a new class, TypeScript says: "Okay, from now on, whenever someone says `new AdminUser()`, I will actually use this *new* class you gave me instead."

Method Decorators

Method decorators sit on top of a method. We use them to intercept a function call, do something, and then satisfy the original call.

1. Basic: Logging

Here we just want to run some code every time a method is called.

```
function Log(target: any, methodName: string, descriptor: PropertyDescriptor) {
  // 1. Save the original function code
  const originalMethod = descriptor.value;

  // 2. Replace the function with our new wrapper
  descriptor.value = function(...args: any[]) {
    console.log(`LOG: Calling ${methodName}`);

    // 3. Call the original function
    const result = originalMethod.apply(this, args);

    return result;
  };
}

class Calculator {
  @Log
  add(a: number, b: number) {
    return a + b;
  }
}

const calc = new Calculator();
calc.add(5, 3);
```

Output:

```
LOG: Calling add
```

Modern TC39 Syntax

You don't edit a "descriptor". You just return the new function you want to use.

```
function Log(originalMethod: Function, context: ClassMethodDecoratorContext) {
  const methodName = String(context.name);

  return function(this: any, ...args: any[]) {
    console.log(`LOG: Calling ${methodName}`);
```

```
    return originalMethod.call(this, ...args);
  };
}
```

Why `.call()` and not `.apply()`?

They do the same thing here! You can absolutely use `return originalMethod.apply(this, args);` if you prefer.

Older code uses `.apply(this, args)` because `args` is an array.

Modern code often uses `.call(this, ...args)` because the spread syntax (`...`) unpacks that array into individual items. Using spread with `.call` is just the modern idiomatic style.

Mechanics: Method decorators don't technically "return" a new object to replace the method. Instead, they edit the `descriptor` object directly. By setting `descriptor.value = function(...)`, we are hard-overwriting the original function code in memory.

2. Modifying the Result

We can even change what the function returns!

```
function MakeUppercase(target: any, name: string, descriptor: PropertyDescriptor) {
  const originalMethod = descriptor.value;

  descriptor.value = function(...args: any[]) {
    // Call original method
    const result = originalMethod.apply(this, args);

    // Modify the result before returning
    if (typeof result === "string") {
      return result.toUpperCase();
    }
    return result;
  };
}

class Greeter {
  @MakeUppercase
  sayHello(name: string) {
    return `Hello, ${name}`;
  }
}
```

```
const greeter = new Greeter();
console.log(greeter.sayHello("World"));
```

Output:

```
HELLO, WORLD
```

Modern TC39 Syntax

```
function MakeUppercase(
  originalMethod: Function,
  context: ClassMethodDecoratorContext
) {
  return function(this: any, ...args: any[]) {
    const result = originalMethod.call(this, ...args);
    if (typeof result === "string") return result.toUpperCase();
    return result;
  };
}
```

Property Decorators

Property decorators are applied to class properties. They receive two parameters: the target object and the property name.

Basic Property Decorator

```
function ReadOnly(target: any, propertyKey: string) {
  console.log(`Property ${propertyKey} is marked as read-only`);
}

class Config {
  @ReadOnly
  apiUrl: string = "https://api.example.com";
```

```
  @ReadOnly
  timeout: number = 5000;
}

const config = new Config();
```

Output:

```
Property apiUrl is marked as read-only
Property timeout is marked as read-only
```

Modern TC39 Syntax

Note: Modern decorators don't work well on plain fields for initializing logic like `ReadOnly`. Instead, we use the `accessor` keyword which generates auto-methods we can intercept.

```
function ReadOnly(target: any, context: ClassAccessorDecoratorContext) {
  return {
    set(value: any) {
      throw new Error(`Cannot write to read-only property ${String(context.name)}`);
    },
    init(initialValue: any) { return initialValue; }
  };
}

class Config {
  @ReadOnly accessor apiUrl = "https://api.example.com";
}
```

Default Value Decorator

```
function Default(value: any) {
  return function(target: any, propertyKey: string) {
    target[propertyKey] = value;
  };
}

class Settings {
```

```
  @Default("English")
  language!: string;

  @Default(true)
  notifications!: boolean;
}

const settings = new Settings();
console.log(settings.language);
console.log(settings.notifications);
```

Output:

```
English
true
```

Modern TC39 Syntax

In modern syntax, we return an `initializer` function to set the starting value of a field.

```
function Default(defaultValue: any) {
  return function(value: undefined, context: ClassFieldDecoratorContext) {
    return function(initialValue: any) {
      if (initialValue === undefined) return defaultValue;
      return initialValue;
    }
  }
}

class Settings {
  @Default("English")
  language?: string; // value passed to decorator is undefined
}
```

Parameter Decorators

Parameter decorators are applied to function parameters. They receive three parameters: the target object, the method name, and the parameter index.

Basic Parameter Decorator

```
function LogParameter(
  target: any,
  propertyKey: string,
  parameterIndex: number
) {
  console.log(`Parameter at index ${parameterIndex} in method ${propertyKey}`);
}

class OrderService {
  createOrder(
    @LogParameter orderId: string,
    @LogParameter amount: number
  ) {
    console.log(`Order ${orderId} for $${amount} created`);
  }
}

const service = new OrderService();
service.createOrder("ORD-123", 99.99);
```

Output:

```
Parameter at index 1 in method createOrder
Parameter at index 0 in method createOrder
Order ORD-123 for $99.99 created
```

Modern TC39 Syntax

Note: Typescript 5.0 / TC39 Limitation

Standard decorators **do not support** parameter decorators yet! This is a major reason why frameworks like Angular still use `experimentalDecorators: true`. If you are using modern decorators, you cannot decorate parameters.

Note: Parameter decorators are executed in reverse order (right to left).

Decorator Factories

A "Factory" is just a function that *returns* another function. We use this when we want to pass arguments to our decorator, like `@Component({ ... })` or `@Path("/users")`.

1. Configurable Class Decorator

```
function Component(options: { selector: string, template: string }) {
  // We return the actual decorator function
  return function(constructor: Function) {
    console.log(`Building component with selector: ${options.selector}`);
    console.log(`Template: ${options.template}`);
  }
}

@Component({
  selector: "user-profile",
  template: "<div>Profile</div>"
})
class UserProfile {}
```

Output:

```
Building component with selector: user-profile
Template: <div>Profile</div>
```

Modern TC39 Syntax

Modern Factory (Class):

```
function Component(options: { selector: string, template: string }) {
  return function(value: Function, context: ClassDecoratorContext) {
    if (context.kind === "class") {
        console.log(`Building component with selector: ${options.selector}`);
        // In reality, you might attach metadata to the class prototype here
    }
  }
}
```

2. Configurable Method Decorator

```
function PayWithMessage(message: string) {
  return function(target: any, name: string, descriptor: PropertyDescriptor) {
    const originalMethod = descriptor.value;

    descriptor.value = function(...args: any[]) {
      console.log(`Payment Protocol: ${message}`);
      return originalMethod.apply(this, args);
    }
  }
}

class Store {
  @PayWithMessage("Processing Credit Card...")
  buyBook() {
    console.log("Book Bought!");
  }

  @PayWithMessage("Counting Cash...")
  buyCandy() {
    console.log("Candy Bought!");
  }
}

const store = new Store();
store.buyBook();
```

Output:

```
Payment Protocol: Processing Credit Card...
Book Bought!
```

Modern TC39 Syntax

Factories work exactly the same way conceptually: A function returning the modern decorator function.

```
function myDecorator(message: string) {
  return function (value: Function, context: ClassMethodDecoratorContext) {
    return function (this: any, ...args: any[]) {
      console.log(message);
      return value.apply(this, args);
    };
  };
}

class Example {
  @myDecorator("Hello!")
  sayHi() {
    console.log("Hi");
  }
}

new Example().sayHi();
```

Output:

```
Hello!
Hi
```

Mechanics: When you write `@PayWithMessage("...")`, you are actually *calling* the factory function immediately.

1. `PayWithMessage("Hi")` runs first and returns the *inner* function.
2. TypeScript takes that inner function and uses it as the actual decorator.

Comparison: Legacy vs Modern Syntax

A quick reference guide to the differences you've seen above.

Feature	Legacy (Experimental)	Modern (TC39 Standard)
TS Confg	`"experimentalDecorators": true`	`"experimentalDecorators": false`
Arguments	`(target, key, descriptor)`	`(value, context)`
Logic	Edit the `descriptor` object	Return the new function/class
Auto-Accessors	Manual `Object.defineProperty`	Native `accessor` keyword
Parameter Decorators	Supported ☑	Not Supported ❌

Decorator Execution Order

When multiple decorators are applied, they execute in a specific order:

```
function First() {
  console.log("First(): factory called");
  return function(target: any, propertyKey: string) {
    console.log("First(): decorator executed");
  };
}

function Second() {
  console.log("Second(): factory called");
  return function(target: any, propertyKey: string) {
    console.log("Second(): decorator executed");
  };
}

class Example {
  @First()
```

```
  @Second()
  method() {}
}

const example = new Example();
```

Output:

```
First(): factory called
Second(): factory called
Second(): decorator executed
First(): decorator executed
```

Important rules:

- Decorator factories are evaluated top to bottom
- Decorator functions are executed bottom to top
- This is similar to function composition in mathematics

Guided Example: The @Confirm Decorator

Create a decorator factory called `@Confirm` that asks for comparison before running a method.

(Since we can't do real user input here easily, we will simulate it with a boolean flag)

```
function Confirm(message: string) {
  return function(target: any, key: string, descriptor: PropertyDescriptor) {
    const original = descriptor.value;

    descriptor.value = function(...args: any[]) {
      const isConfirmed = confirm(message); // Browser popup

      if (isConfirmed) {
        return original.apply(this, args);
      } else {
        return null; // Cancelled
      }
```

```
    };
  };
}

class BankAccount {
  balance = 1000;

  @Confirm("Are you sure you want to withdraw money?")
  withdraw(amount: number) {
    this.balance -= amount;
    console.log(`Withdrawn ${amount}. New balance: ${this.balance}`);
  }
}

// In a real browser, this would pop up a dialog!
const account = new BankAccount();
account.withdraw(100);
```

Modern TC39 Syntax

Modern Implementation:

```
function Confirm(message: string) {
  return function(originalMethod: Function, context: ClassMethodDecoratorContext) {
    return function(this: any, ...args: any[]) {
      const isConfirmed = confirm(message);

      if (isConfirmed) {
        return originalMethod.call(this, ...args);
      }
      return null;
    }
  }
}
```

Note: In a Node.js console environment, `confirm()` doesn't exist, but this logic perfectly illustrates how we use decorators to "guard" methods.

Decorators and Metadata in Angular Context

Angular heavily uses decorators to add metadata to classes. This metadata tells Angular how to process and use those classes.

Understanding @Component

The `@Component` decorator marks a class as an Angular component and provides configuration metadata.

```
// Simplified example of how @Component works conceptually

function Component(metadata: {
  selector: string;
  template: string;
  styleUrls?: string[];
}) {
  return function(constructor: Function) {
    // Angular stores this metadata internally
    (constructor as any).__metadata__ = metadata;
    console.log(`Component "${metadata.selector}" registered`);
  };
}

@Component({
  selector: "app-greeting",
  template: "
<h1>Hello {{ name }}</h1>
",
  styleUrls: ["./greeting.component.css"]
})
class GreetingComponent {
  name: string = "World";

  constructor() {
    console.log("GreetingComponent created");
  }
```

```
}

const greeting = new GreetingComponent();
```

Output:

```
Component "app-greeting" registered
GreetingComponent created
```

Key concepts:

- `selector` : The HTML tag name for the component
- `template` : The HTML template for the component
- `styleUrls` : Array of CSS file paths for styling

Understanding @Injectable

The `@Injectable` decorator marks a class as available for dependency injection in Angular.

```
// Simplified example of how @Injectable works conceptually

function Injectable(config?: { providedIn: string }) {
  return function(constructor: Function) {
    (constructor as any).__injectable__ = true;
    (constructor as any).__providedIn__ = config?.providedIn || "root";
    console.log(`Service ${constructor.name} is injectable`);
  };
}

@Injectable({
  providedIn: "root"
})
class UserService {
  private users: string[] = ["Alice", "Bob"];

  getUsers(): string[] {
    return this.users;
  }

  addUser(name: string): void {
```

```
    this.users.push(name);
  }
}

const userService = new UserService();
console.log(userService.getUsers());
```

Output:

```
Service UserService is injectable
[ 'Alice', 'Bob' ]
```

Key concepts:

- `providedIn: 'root'` : Makes the service available application-wide
- Services marked with `@Injectable` can be injected into components and other services
- Angular's dependency injection system manages the lifecycle of these services

Combining Decorators in Angular

```
// Conceptual example showing multiple decorators together

@Component({
  selector: "app-user-list",
  template: `


      {{ user }}


  `
})
class UserListComponent {
  users: string[] = [];

  constructor(private userService: UserService) {
    this.users = this.userService.getUsers();
    console.log("UserListComponent initialized with users:", this.users);
  }
```

```
}

const component = new UserListComponent(userService);
```

Output:

```
UserListComponent initialized with users: [ 'Alice', 'Bob' ]
```

Student Exercises

Now it's your turn. Try to implement these scenarios to solidify your understanding.

Output:

Strategy: How to Solve These

When writing a method decorator, follow this 5-step mental checklist:

1. **Identify the Target:** Are you decorating a class, method, or property? This determines your arguments (e.g., `target, key, descriptor`).
2. **Capture the Original:** create a variable (e.g., `const original = descriptor.value`) to hold the old function.
3. **Create the Wrapper:** Write a `function(...args) { ... }` that contains your new logic.
4. **Run the Original:** Inside your wrapper, use `original.apply(this, args)` to run the actual code.
5. **Return/Replace:** Assign your wrapper back to `descriptor.value`.

Exercise 1: The `@Deprecated` Warning

Goal: Create a method decorator that prints "WARN: This method is old!" to the console every time the method is used, but still runs the code.

Need a Hint?

Just like the Logging example, you need to wrap the original method. Add a `console.warn()` line before calling `originalMethod.apply()`.

Exercise 2: The `@Timing` Decorator

Goal: Write a decorator that measures how long a method takes to run using `performance.now()` (or `Date.now()`).

Need a Hint?
1. Get start time: `const start = Date.now();`
2. Run method: `const result = original.apply(this, args);`
3. Get end time: `const end = Date.now();`
4. Log difference: `console.log(end - start);`
5. Return result.

Exercise 3: The `@Sealed` Class Decorator

Goal: Use `Object.seal(constructor)` and `Object.seal(constructor.prototype)` to prevent anyone from adding new properties to your class at runtime.

Need a Hint?
This is a Class Decorator. It takes `constructor` as an argument. You just need to run `Object.seal(constructor)` inside the decorator function body.

Exercise 4: The `@Repeat(n)` Factory

Goal: Create a Factory that takes a number (e.g., `@Repeat(3)`) and runs the method that many times when called once.

Need a Hint?
1. Create a function `Repeat(times: number)`.
2. Return the actual decorator function inside it.
3. Inside the decorator's wrapper, write a simple `for` loop from 1 to `times`.
4. Call `original.apply()` inside that loop.

Summary

In this module, you learned:

- What decorators are and how they enhance classes and members
- Class decorators for modifying class behavior
- Method decorators for wrapping method execution
- Property decorators for adding metadata to properties
- Parameter decorators for tracking method parameters
- Decorator factories for creating configurable decorators
- Decorator execution order and composition
- How Angular uses `@Component` and `@Injectable` decorators

Decorators are a powerful feature that enables declarative programming and metadata-driven frameworks like Angular. They help separate concerns and make code more maintainable. Practice creating custom decorators to become comfortable with this advanced TypeScript feature.